



Generative AI Applications in SE Lab 03

Submitted to

Dr. Humaira Waqas

Submitted by

M Ehtisham Amjad (231996)

BSSE-VI-B

Graded Task 1: Input Validation Enhancement

Objective

Extend the Code node validation to reject requests where `studentName` contains non-alphabetic characters and `question` is fewer than 10 characters.

Implementation

The validation Code node was updated with two additional rules:

- `studentName` must match the regex `/^[a-zA-Z\s]+$/` — rejects names with numbers or symbols
- `question` must be at least 10 characters long

If either rule fails, a descriptive message is added to the `missingFields` array and `isValid` is set to `false`, routing the request to the 400 FALSE branch.

Test Case 1 — Invalid `studentName`

Request sent:

```
{
  "studentName": "Ehtisham123",
  "topic": "Machine Learning",
  "question": "What is the difference between supervised and unsupervised?"
}
```

Expected Result: 400 Bad Request **missingFields** should contain: "studentName contains invalid characters"

The screenshot shows a REST client interface with a POST request to `http://localhost:5678/webhook-test/lab03/submit`. The request body is a JSON object: `{ "studentName": "Ehtisham123", "topic": "Machine Learning", "question": "What is the difference between supervised and unsupervised learning?" }`. The response is a 400 Bad Request with a status of 73 ms and 624 B. The response body is a JSON object: `{ "status": "error", "message": "Missing required fields", "missingFields": [ "studentName contains invalid characters" ] }`.

Request sent:

```
{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What?"
}
```

Expected Result: 400 Bad Request missingFields should contain: "question too short"

The screenshot shows the Postman interface for a POST request to `http://localhost:5678/webhook-test/lab03/submit`. The request body is a JSON object with `studentName`, `topic`, and `question` fields. The response status is **400 Bad Request** with a message indicating that the `question` field is missing or too short.

Request Body (JSON):

```
{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What?"
}
```

Response Body (JSON):

```
{
  "status": "error",
  "message": "Missing required fields",
  "missingFields": [
    "question too short - minimum 10 characters required"
  ]
}
```

Test Case 3 — Valid Request**Request sent:**

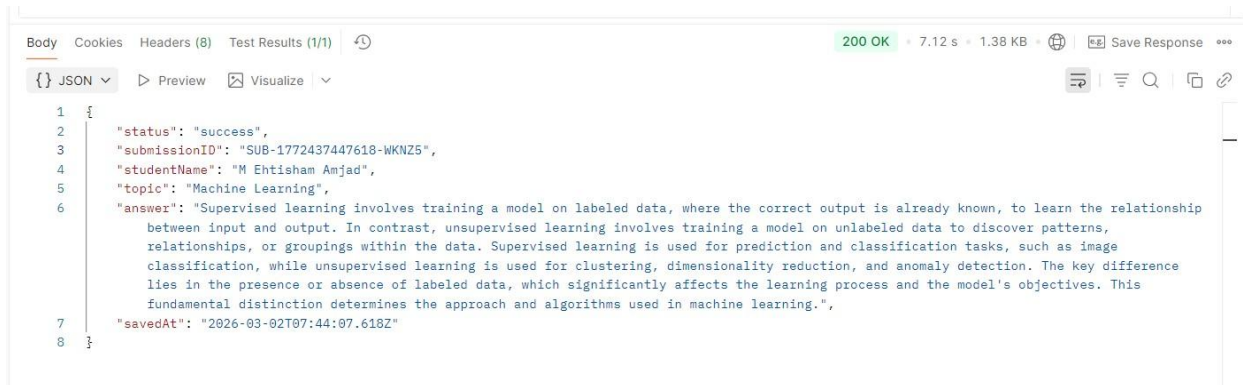
```
{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What is the difference between supervised and unsupervised learning?"
}
```

Expected Result: 200 Success with AI-generated answer.

The screenshot shows the Postman interface for a POST request to `http://localhost:5678/webhook-test/lab03/submit`. The request body is a JSON object with `studentName`, `topic`, and `question` fields. The response status is **200 Success**.

Request Body (JSON):

```
{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What is the difference between supervised and unsupervised learning?"
}
```



Graded Task 2: Duplicate Submission Detection

Test Case 1 — First Submission (New)

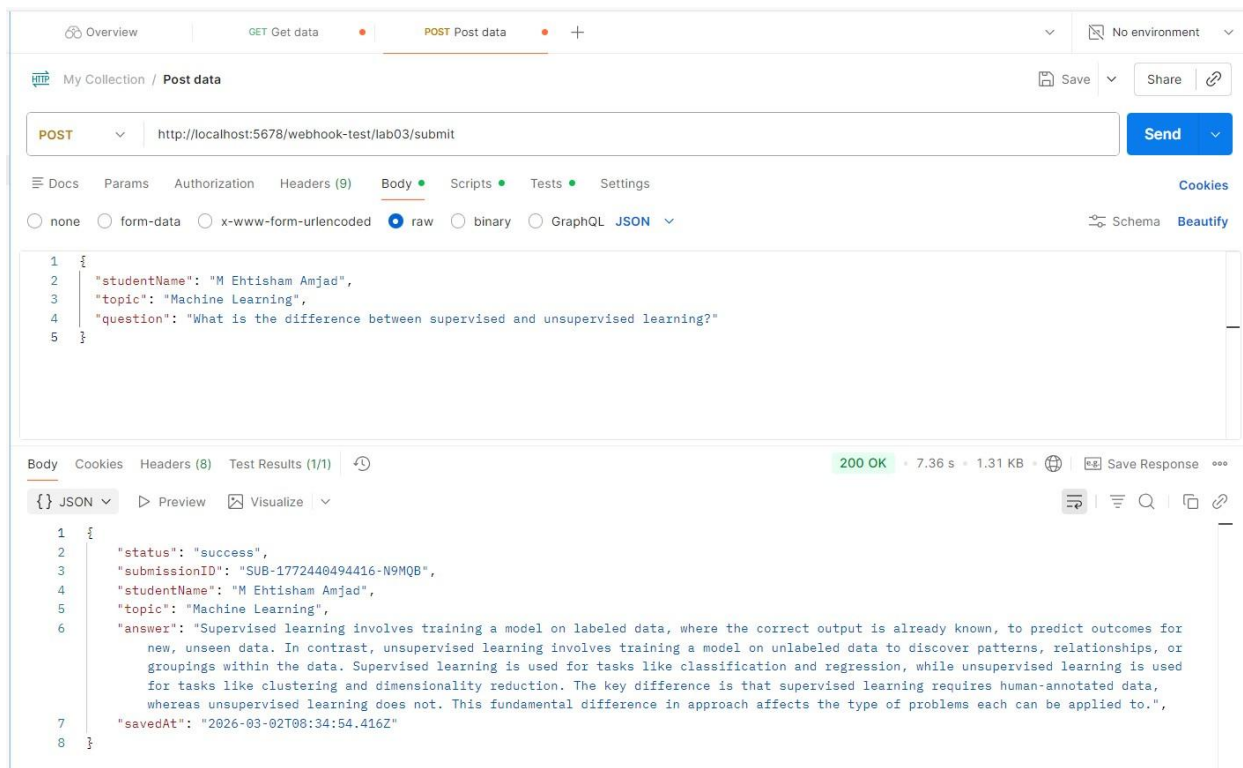
Request sent:

```

{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What is the difference between supervised and unsupervised learning?"
}

```

Expected Result: 200 Success — saved to sheet



Test Case 2 — Duplicate Submission

Exact same request sent again.

Expected Result: 409 Conflict

```
{
  "status": "duplicate",
  "message": "This question was already submitted by this student"
}
```

The screenshot shows a REST client interface with a POST request to `http://localhost:5678/webhook-test/lab03/submit`. The request body is a JSON object:

```
{
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "question": "What is the difference between supervised and unsupervised learning?"
}
```

The response is a 409 Conflict status with the following JSON body:

```
{
  "status": "duplicate",
  "message": "This question was already submitted by this student"
}
```

Lab03-SubmissionsDB

SubmissionID	StudentName	Topic	Question	AIAnswer	Status	Timestamp
SUB-177244049	M Ehtisham Amj	Machine Learnin	What is the diffe	Supervised learn	accepted	2026-03-02T08:34:54.416Z
SUB-177244069	M Ehtisham Amj	Machine Learnin	What is overfittir	Overfitting occur	accepted	2026-03-02T08:38:15.905Z

Graded Task 3: Workflow Export & API Documentation

Workflow Export

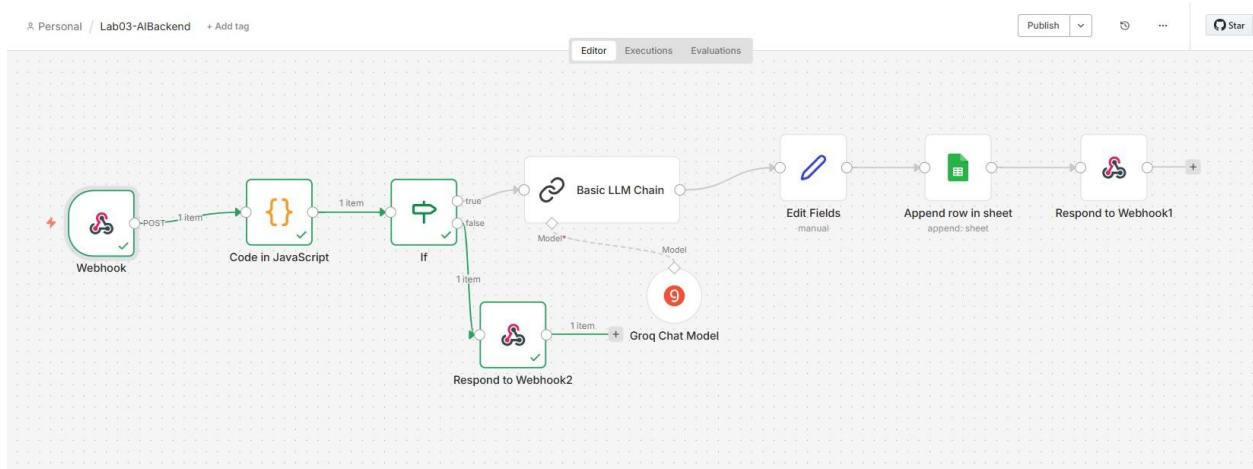


Figure 1: Task 1 — Invalid Request (400 FALSE Branch)

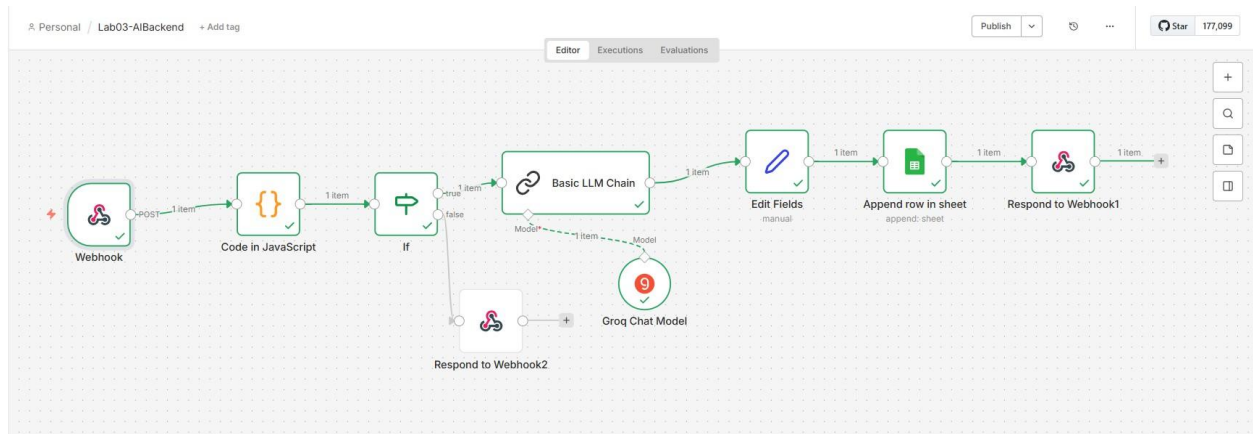


Figure 2: Task 1 — Valid Request (200 TRUE Branch)

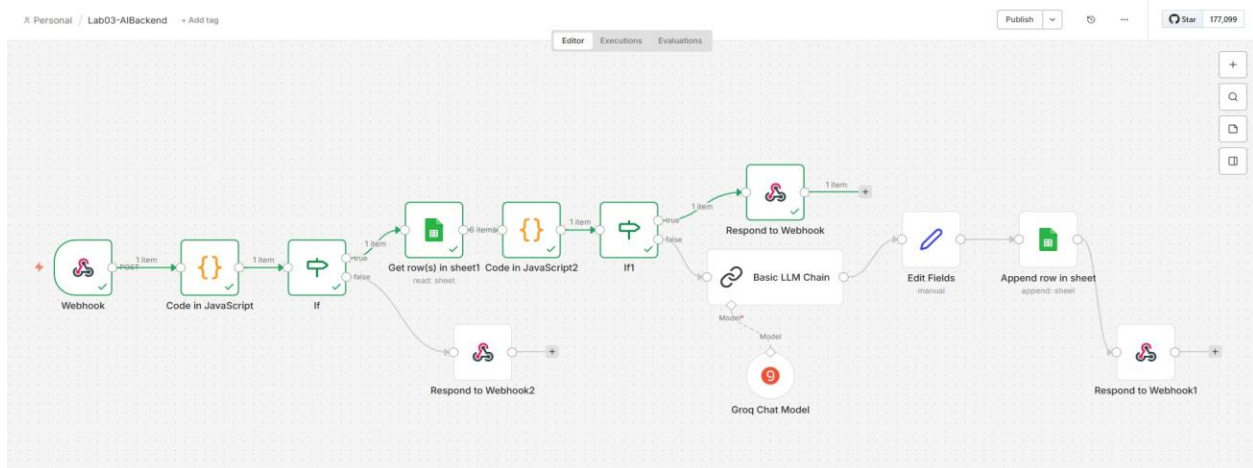


Figure 3: Task 2 — Duplicate Detected (409 Branch)

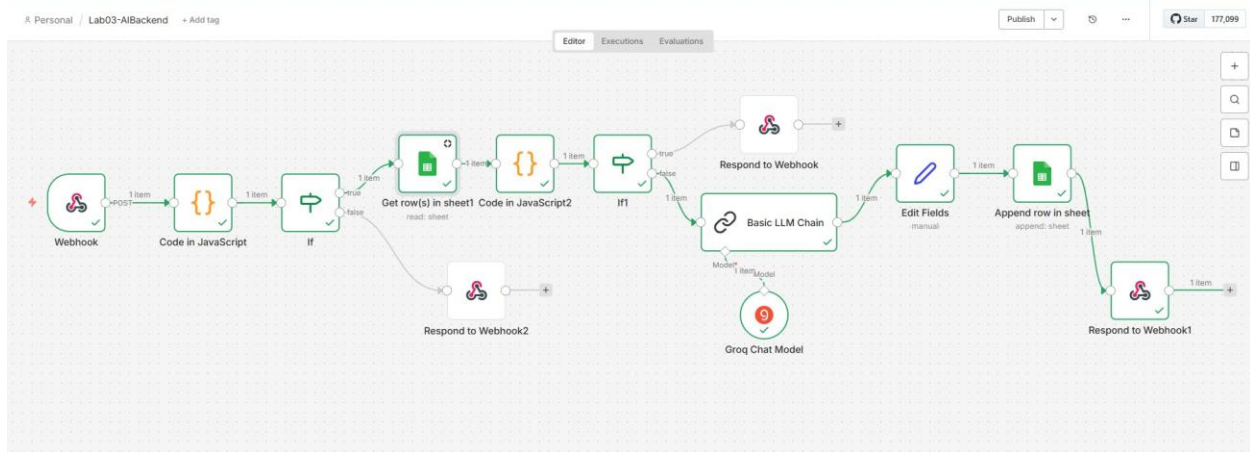


Figure 4: Task 2 — Complete Duplicate Detection Workflow (New Submission, 200 Success)

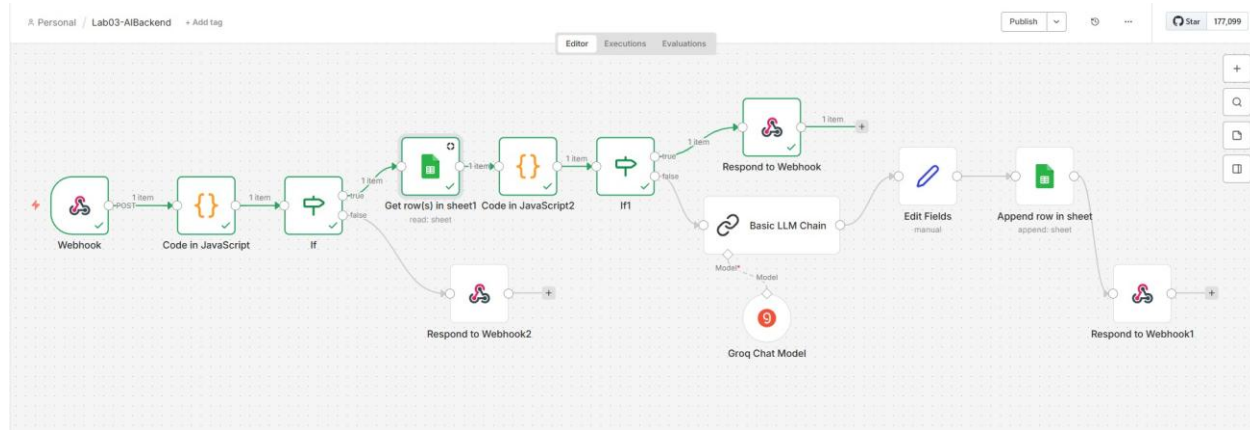


Figure 5: Task 2 — First Submission (200 FALSE Branch)

API Documentation

Endpoint Table

Path	Method	Description
/webhook-test/lab03/submit	POST	Submit a student question. Validates payload, checks duplicates, generates AI answer, saves to sheet, returns JSON
/webhook-test/lab03/submissions	GET	Query submissions by ?studentName=. Returns matching rows as JSON array

Request Schema

POST /lab03/submit — Body (JSON):

Field	Type	Required	Validation Rules
studentName	String	Yes	Letters and spaces only. No numbers or special characters
topic	String	Yes	Must not be empty
question	String	Yes	Must not be empty. Minimum 10 characters

GET /lab03/submissions — Query Parameter:

Parameter	Type	Required	Description
studentName	String	No	Filter by student name. If omitted, all rows returned

Response Schema**200 — Success (POST):**

json

```
{
  "status": "success",
  "submissionID": "SUB-1772440695905-RVHSR",
  "studentName": "M Ehtisham Amjad",
  "topic": "Machine Learning",
  "answer": "Supervised learning uses labelled data...",
  "savedAt": "2026-03-02T08:38:15.905Z"
}
```

409 — Duplicate:

json

```
{
  "status": "duplicate",
  "message": "This question was already submitted by this student"
}
```

400 — Validation Error:

json

```
{
  "status": "error",
  "message": "Missing required fields",
  "missingFields": ["question too short", "studentName contains invalid characters"]
}
```

}

200 — GET Submissions:

json

```
{
  "status": "success",
  "studentName": "M Ehtisham Amjad",
  "count": 2,
  "submissions": [
    {
      "SubmissionID": "SUB-...",
      "StudentName": "M Ehtisham Amjad",
      "Topic": "Machine Learning",
      "Question": "What is the difference...",
      "AIAnswer": "Supervised learning...",
      "Status": "accepted",
      "Timestamp": "2026-03-02T08:34:54.416Z"
    }
  ]
}
```

Known Limitations

1. **No authentication** — the endpoint has no API key or token. Anyone with the URL can send requests.
2. **Sheet-based duplicate check is slow** — every POST request reads the entire sheet to check for duplicates. Performance degrades as rows increase since there is no indexing.
3. **Test URL only works in listen mode** — /webhook-test/ only works while the workflow is actively listening. Production deployment requires /webhook/ with the workflow activated.
4. **No pagination on GET** — all matching rows are returned at once. With a large dataset this could cause slow responses or timeouts.

Reflection

The n8n workflow mirrors the architecture of an Express.js backend exactly — the Webhook node is the route handler, the Code node is validation middleware, Google Sheets replaces the database layer, and Respond to Webhook closes the request/response cycle. The key advantage of n8n is speed: what would require setting up a Node.js project, writing route handlers, connecting a database, and deploying a server was built visually in a single lab session. However, n8n lacks critical production features — there is no authentication middleware, no support for database transactions or relational queries, no unit testing, and complex logic becomes harder to maintain visually than in clean code. Additionally, scaling beyond a few concurrent users would require a

proper server. For academic prototyping n8n is excellent, but a production system serving real users would require a proper backend framework such as Express.js or Django with a real database like PostgreSQL.

End Lab 03